

Name: Farrel Anggito Baswara Marpaung
NIM: 25/559096/PA/23506

Image Sample

- Three image will become the sample with resolution of 256x256
- Geometric.png will helps reveal edge blur and edge preservation



- Sunset.png will reveal overall visual quality under realistic conditions



- Noisy.png with salt and pepper noise will show the difference between each method



Algorithm Design

1. Down Sampling

1.1 Down Sampling MAX

Max downsampling uses the largest value in each color channel of a block. It favors bright components and can suppress darker details. Because the maxima are selected independently, the output RGB triplet does not have to match any one original pixel.

1. Read the input dimensions and calculate height, factor and width, factor.
2. Create a uint8 array with the reduced dimensions and the original channel count.
3. Select the corresponding factor x factor block for each output position.
4. Apply `np.max(block, axis=(0, 1))` to obtain the channel-wise maxima.
5. Store the values and repeat until every output pixel has been calculated.

1.2 Down Sampling Median

Median downsampling represents each block by its central intensity values, reducing the influence of isolated extremes. With a 2 x 2 block there are four samples per channel: NumPy averages the second and third values in sorted order [3]. It does not simply select one middle pixel.

1. Calculate the reduced dimensions and allocate the uint8 output array.
2. Extract the non-overlapping 2 x 2 block associated with the output location.
3. Determine the median of its four values independently for each channel.
4. Assign the result to uint8, any fractional part is truncated by this code.
5. Continue across all blocks and return the lower-resolution image.

1.3 Down Sampling Average

Average downsampling computes the arithmetic mean of all four samples in each channel. Every input value contributes equally. This is equivalent to a 2 x 2 box average followed by decimation on the chosen grid. It smooths local variations, including real detail and noise, but does not eliminate all possible aliasing.

1. Calculate the output dimensions and create the uint8 result array.
2. Use the output row and column to locate the corresponding input block.
3. Apply `np.mean(block, axis=(0, 1))` independently to the RGB channels.
4. Store the floating-point mean in uint8, truncating its fractional part.
5. Repeat for every block and return the downsampled image.

2. Upsampling

2.1 Nearest Neighbour Upsampling

The function copies a source pixel without mixing it with neighboring values. At scale 2, integer truncation maps each pair of output rows and columns to one source pixel, producing a 2 x 2 repeated block. Although named nearest neighbor, this implementation uses floor-based replication for nonnegative coordinates, rather than explicitly rounding $x_{\text{new}} / \text{scale}$ to the closest integer.

1. Read the source dimensions and multiply height and width by scale.
2. Allocate the enlarged uint8 output array and iterate over its pixels.
3. Compute $y_{\text{old}} = \text{int}(y_{\text{new}} / \text{scale})$ and $x_{\text{old}} = \text{int}(x_{\text{new}} / \text{scale})$.
4. Copy `image[y_old, x_old]` directly into the output RGB pixel.
5. Repeat until the enlarged image is filled; no new intensity values are interpolated

2.2 Bilinear Upsampling

Bilinear interpolation estimates an output pixel from the four source samples surrounding its mapped position. Horizontal and vertical fractional distances determine the weights. The resulting value changes smoothly between neighboring samples. The weights are nonnegative and sum to one, so the interpolation itself stays within the range of the contributing channel values.

1. Create the enlarged output and map each output location to fractional source coordinates.
2. Find the floor indices x_1 and y_1 , then obtain x_2 and y_2 ; clamp the upper neighbors to the image boundary.
3. Compute $dx = x_{\text{old}} - x_1$ and $dy = y_{\text{old}} - y_1$, and convert the four RGB samples to float.
4. Combine the samples with weights $(1-dx)(1-dy)$, $dx(1-dy)$, $(1-dx)dy$, and $dx dy$.
5. Clip the result to 0-255, assign it to uint8, and repeat for all output pixels.

2.3 Bicubic Upsampling

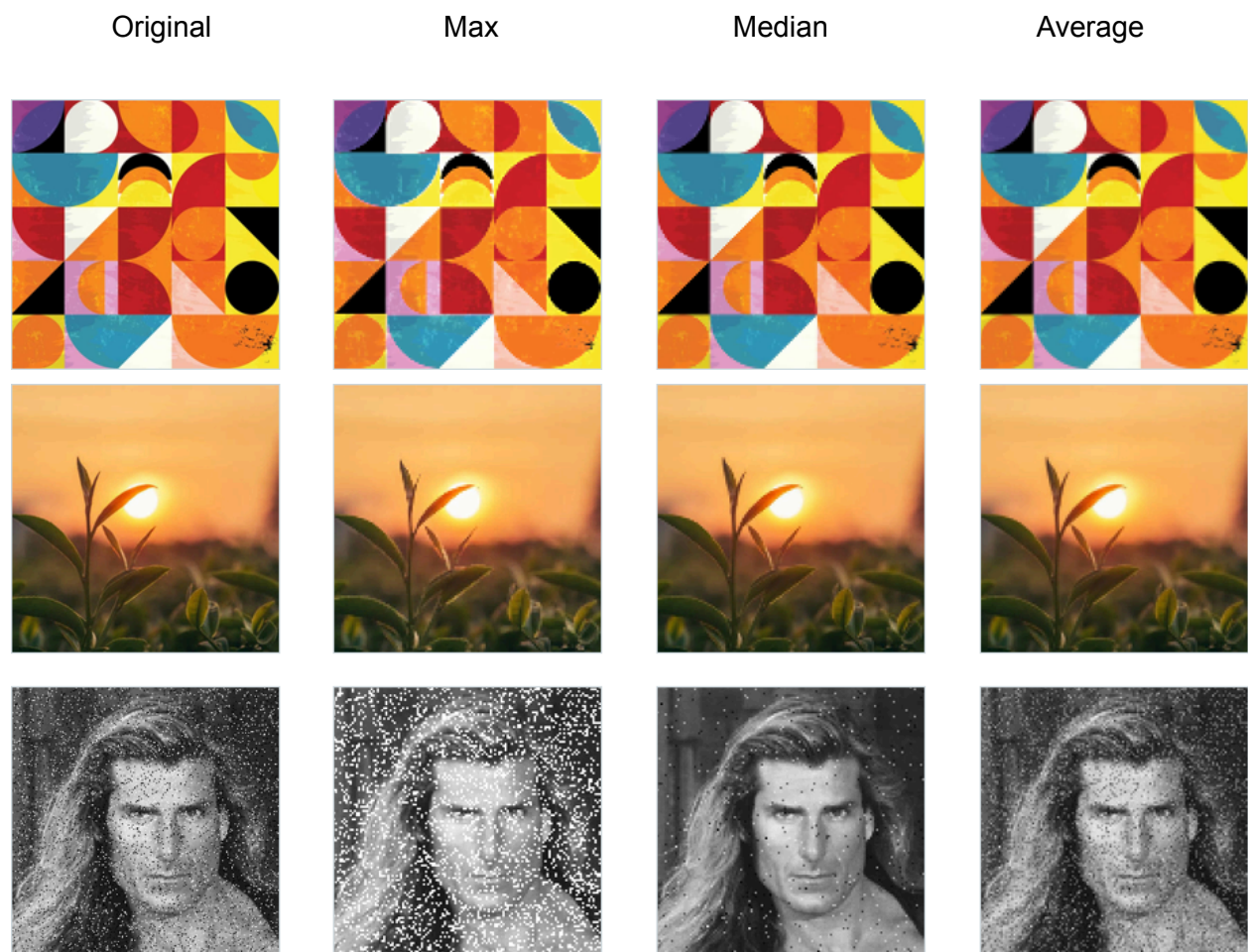
The submitted bicubic function uses a separable cubic-convolution kernel with $a = -0.5$. Each output location considers a 4 x 4 source neighborhood. Horizontal and vertical kernel weights are multiplied, then used to accumulate an RGB value. A wider neighborhood can provide a sharper transition than bilinear interpolation, but negative kernel weights can also produce overshoot near contrast boundaries [1].

1. Allocate the enlarged uint8 image, map each output location to source coordinates, and find its floor indices.
2. Visit the 16 neighbors defined by offsets -1, 0, 1, and 2 in both directions.

3. Clamp sample indices to valid image bounds, but calculate kernel distances from the original, unclamped neighbor coordinates.
4. Multiply the horizontal and vertical cubic weights, accumulate weighted float RGB values, and accumulate total_weight.
5. Divide by total_weight when nonzero, clip to 0-255, store as uint8, and repeat.

Analysis

1. Down Sampling



1.1 Max downsampling.

In Geometric, light-colored regions dominate mixed blocks, which can shrink dark shapes and alter the apparent contour of the black circle and diagonals. This is brightness selection, not reliable edge preservation. In Sunset, the sun and illuminated leaf

boundaries remain prominent, but bright values can replace nearby dark detail. In Noisy, a single white impulse can dominate a block, producing many bright spots and a substantial brightness increase. Max therefore responds strongly to salt noise and is unsuitable for faithful reduction of this noisy image.

1.2 Median downsampling.

In Geometric, the median keeps the main color regions recognizable, although curved boundaries become coarser and some fine textures disappear. A block split equally between two intensities can produce an intermediate value. In Sunset, the sky and larger leaf structures remain close to the original, while thin details are reduced. In Noisy, median visibly removes many isolated black and white speckles and makes facial and hair structures clearer than average or max. Residual spots remain because multiple corrupted samples within a small block can affect its median.

1.3 Average downsampling.

In Geometric, averaging blends colors in blocks that cross boundaries, softening curves, diagonals, and small textures. In Sunset, it preserves broad orange gradients and gives a balanced reduction, while slightly softening the sun and thin stems. In Noisy, extreme impulses are diluted into intermediate intensities, so individual black and white points become less stark, but a granular pattern remains across the face and background. Average is useful for smooth photographic content, for the impulse noise in this sample, median gives the cleaner visual result.

2. Upsampling

Original

Nearest

Bilinear

Bicubic



2.1 Nearest neighbor upsampling.

In Geometric, the nearest neighbor repeats each low-resolution pixel as a 2×2 block, making the circle and diagonal boundaries look stepped when enlarged. In Sunset, large gradients remain recognizable, while thin leaf edges appear coarse. In Noisy, it duplicates the residual fluctuations already produced by average downsampling, creating blocky patches. It adds no interpolation blur, but the earlier averaging has already removed detail. Its unchanged low-resolution values explain why the overall composition can look almost identical to the low-resolution subplot.

2.2 Bilinear upsampling.

In Geometric, bilinear reduces abrupt block transitions but introduces a wider band of mixed colors around contrasting shapes. In Sunset, the sky looks continuous and leaf contours appear smoother, with reduced local sharpness. In Noisy, residual grain becomes softer and spreads between neighboring samples, but it is not removed as effectively as by median downsampling. The four-sample interpolation explains the smoother appearance. This smoothness should not be interpreted as recovery of the original edge positions, fine textures, or clean facial detail.

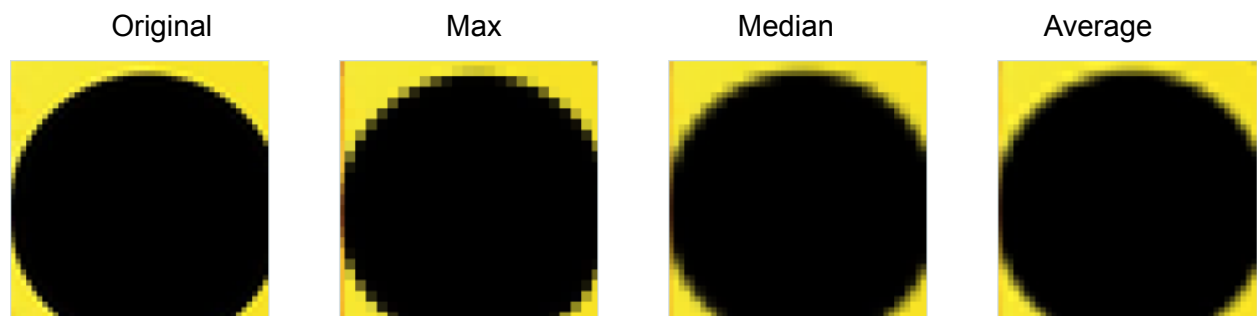
2.3 Bicubic upsampling.

In Geometric, bicubic gives a more defined transition than bilinear in many boundary regions, while retaining smoother contours than nearest neighbor. Its negative weights can introduce narrow overshoot bands near strong contrast. In Sunset, some leaf boundaries look slightly crisper than in the bilinear image, although fine detail remains limited by the average-downsampled input. In Noisy, local contrast can also make residual grain more noticeable than with bilinear. Bicubic provides a different sharpness smoothness trade off, it is not automatically the most accurate or best denoising result.

2.4 Why the full image look similar

Whole-image comparisons can hide the changes because the scene, colors, and large structures remain the same. The notebook also displays low and high resolution arrays in similarly sized subplot areas, and Matplotlib may resample those arrays for display [4].

Geometric boundary cropping:



Nearest neighbor exposes repeated square pixels. Bilinear distributes the color transition over nearby pixels. Bicubic changes the transition profile using its wider, partly negative kernel. At an integer source location, all three reproduce the same source sample, so their differences are concentrated mainly at newly inserted positions and near boundaries.

3. Conclusion

Downsampling changes what information is retained. Max favors bright channel values and strongly emphasizes salt noise. Median rejects many isolated extremes and gives the cleanest visual result for the noisy sample, although it can discard fine features. Average gives a balanced reduction of the sunset photograph and smooths small variations, while blending boundaries and retaining diluted impulse noise.

Upsampling changes how the retained samples are represented. Nearest neighbor preserves source values with visible blocks; bilinear smooths transitions and softens edges; bicubic can give crisper transitions but can also accentuate noise or introduce overshoot. All three return the image to 256 x 256 without restoring the exact information lost in the 128 x 128 representation.